Two episodes established that a capability in a parameter list makes an effect visible, and that this survives higher-order functions. But visible at what resolution? Knowing a function was handed a Database tells you it can touch the database — not whether it only reads, or also writes; not whether it needs the database to itself, or can share it. Presence is a coarse grain. This episode is about giving effects a finer one, and about the single awkward case that decides how.

The Logger problem

Here is the case that a two-point system can't express:

```
fn read_users(db: &borrowed Database) -> Vec[User]       // read
fn save_order(db: &mut borrowed Database) -> unit        // write, exclusive
fn log_info(logger: &borrowed Logger, msg: &borrowed str)  // write — but shareable
```

The first two are comfortable. Reading users needs a shared, read-only handle; saving an order needs exclusive write access. Those map cleanly onto the two borrow forms most languages give you: & for shared reads, &mut for exclusive writes.

The third breaks the mapping. A logger is written to — logging appends a line — so it isn't a read. But it does not want to be exclusive: the whole point of a logger is that many callers, even many threads, write to it at once, with the safety handled inside (a lock, a lock-free queue). Pass it as

Two axes that Rust ties together

The reason there's no word is that & and &mut are each doing two jobs at once. & means shared and read-only; &mut means exclusive and writable. Two independent questions — may several borrowers hold this at once? and may a borrower mutate through it? — have been fused into a single either/or. That fusion is convenient and it covers most code, but it leaves one of the four combinations unnameable: shared and writable. Which is exactly a logger.

Mere's response follows straight from a habit the series keeps showing: when one name is quietly carrying two meanings, split it. So the borrow annotation becomes a grid of two orthogonal axes — sharing (shared / exclusive) and operation (read / write) — and the four points each get a

Why put it in the type at all

There were cheaper answers. The alternatives considered ranged from "keep a single & and let each capability's API decide what's allowed" to attaching marker tags like Logger<Write, Shared> to the capability type. The single-borrow option is the simplest to build — but it pushes the concurrency question entirely onto the capability's implementation, where the type system can't see it: whether cache.set through a shared handle is safe becomes a promise the author makes, not a fact the compiler checks. For a language whose whole premise is maximizing what can be machine-verified, delegating concurrency safety to an unchecked convention is the wrong trade, so that option was rejected outright.

The marker-tag approach can express everything the grid can and more —

What it costs at runtime: nothing

The narrowing was done on paper, then dropped into the compiler as a few implementation slices — and the most telling result is what didn't have to change. The four modes are contextual keywords after &; they ride along in the type as a borrow_mode, and unification treats them as distinct with no subtyping, so handing a &R Database where a &mut R Database is wanted is a type error caught at the call site. But the runtime and all three backends (C, LLVM, Wasm) simply ignore the mode: a borrow is a pointer regardless of its mode, so the generated code is identical and feature parity across backends held without touching code generation at all.

That is the same shape as view types being Trivial: a guarantee that lives entirely in the type system and evaporates at runtime. The distinction

What's deferred to the concurrency chapters

Naming &shared write raises a question it doesn't yet answer: what does a type have to prove to be borrowed that way? A shared-writable capability must be internally safe against concurrent access, and there should be a way for the type to declare that it is — a constraint in the spirit of Send/Sync. And the exact code generation for concurrent backends — where the locks actually go, how write ordering is guaranteed — is left until there is a concurrent backend to generate for. Both are deferred on purpose, to the part of the story where concurrency becomes real rather than anticipated. With this, the effect model can say not just what a function may touch but how — read or write, alone or shared. What remains is the ergonomic problem the first effects episode flagged: a realistic function needs